Zurich University of Applied Sciences
Department School of Life Sciences and Facility Management
Institute of Computational Life Sciences

CO3 Project Report – Spring Semester 2026

Fair Water Distribution in Rural Nepal: A Comparison of Lagrange–Newton and Simulated Annealing

Authors:
Triantafyllia Giora
Spyridon Margomenos
Nicolas Biner
Robin Giacomelli

Lecturer:
Prof. Dr. Thomas Ott

Module:
CO3 – Optimisation and
Bio-Inspired Algorithms

Submitted on April 19, 2026

Study program: MSc Applied Computational Life Sciences

Chapter 1

Fair Water Distribution in Rural Nepal

1.1 Introduction

Access to safe drinking water in rural Nepal remains persistently constrained by seasonal spring depletion, ageing gravity-fed infrastructure, and unequal access between households. Gautam and Dahal [1] document that roughly half of Nepal's community water schemes are non-functional or under-performing, with weak cost recovery, climate-driven source depletion, and institutional fragmentation as recurring obstacles. On the technical side, Laporte et al. [2] formulate the post-2015 earthquake restoration of Nepalese gravity-fed networks as a hierarchical location-allocation and Steiner-forest problem, solved with a two-phase matheuristic. Both works frame *infrastructure* and *coverage*; the complementary operational question of *how to allocate a limited communal supply once the network is in place* has received less formal attention.

In this project we take a single village-scale view: given a fixed daily spring yield and a fixed set of households, how should the source be split day-by-day? We formulate this as a small-scale convex quadratic programme with equity weights and pipe-loss coupling, and we compare two methods from different classes of the CO3 taxonomy [3]: a gradient-based *Lagrange–Newton* solver (Chapters 3–4 of the module) and the stochastic *Simulated Annealing* metaheuristic (Chapter 7). The problem is self-formulated, parameterised from Nepal-specific sources [4, 5], and serves as a compact benchmark on which the two methods can be compared on accuracy, efficiency and robustness.

1.2 Problem Formulation

Variables and data. Let x_i denote the daily water volume delivered to household $i \in \{1, \dots, n\}$, d_i its demand, b_i the WHO-aligned minimum allocation, $a_i \geq 1$ a household-specific delivery-loss factor (so $a_i x_i$ is the source-side draw required to deliver x_i at the tap), and $w_i > 0$ a vulnerability weight that raises the penalty for shortages at prioritised households. S is the total daily spring yield. Values (Table 1.1) follow Nepal's Rural Water Supply Design Standards [4] and the WHO/JMP guidelines [5].

Table 1.1: Parameters of the base scenario.

Parameter	Value / range	Source
Households n	20	typical rural system size
Demand d_i	225 L/day	SUS 45 LPCD $\times$ 5 persons [4]
Minimum b_i	100 L/day	WHO 20 LPCD $\times$ 5 persons [5]
Loss factor a_i	[1.00, 1.25]	pipe / terrain overhead
Vulnerability w_i	[1.00, 1.80]	equity prioritisation
Total supply S	3500 L/day	scarcity scenario [1]

Objective. We minimise the weighted sum of squared deviations from demand,

$$f(\mathbf{x}) \;=\; \sum_{i=1}^{n} w_i \big(d_i - x_i\big)^2. \tag{1.1}$$

The quadratic form penalises large under-deliveries super-linearly, and the weights w_i pull the allocation towards vulnerable households — an equity mechanism absent from a uniformly weighted L_1 objective

$\sum |d_i - x_i|$. Compared to a Rawlsian max–min criterion [6], the weighted L_2 form is smooth and strictly convex, enabling gradient-based solvers and a unique closed-form solution (1.5); the Rawlsian alternative is left as a non-smooth extension for future work.

Constraints. Mass balance at the source (equality) and box bounds per household:

$$\sum_{i=1}^{n} a_i\, x_i \;=\; S, \tag{1.2}$$

$$b_i \;\le\; x_i \;\le\; d_i, \quad i = 1, \ldots, n. \tag{1.3}$$

The upper bound $x_i \le d_i$ prevents over-delivery, and together with $b_i > 0$ subsumes non-negativity. The problem is feasible iff $\sum_i a_i b_i \le S \le \sum_i a_i d_i$; for the base scenario this interval is [2250, 5062.5] L/day, and $S = 3500$ L/day sits strictly inside.

Characterisation. (1.1) is smooth and strictly convex (Hessian $\nabla^2 f = 2\,\mathrm{diag}(\mathbf{w}) \succ 0$); the feasible set defined by (1.2)–(1.3) is a non-empty, compact, convex polytope. The problem therefore admits a unique global minimum fully characterised by the Karush–Kuhn–Tucker (KKT) conditions [7, Ch. 12]. This property allows us to treat the Lagrange–Newton output as a ground-truth reference for validating SA.

1.3 Methods

1.3.1 Lagrange–Newton (trust-constr)

With a multiplier $\lambda \in \mathbb{R}$ for (1.2) and $\boldsymbol{\mu}, \boldsymbol{\nu} \in \mathbb{R}^n_{\ge 0}$ for the lower and upper box bounds, the Lagrangian is

$$\begin{aligned} \mathcal{L}(\mathbf{x}, \lambda, \boldsymbol{\mu}, \boldsymbol{\nu}) \;&= f(\mathbf{x}) - \lambda\big(\textstyle\sum_i a_i x_i - S\big) \\ &\quad - \textstyle\sum_i \mu_i (x_i - b_i) - \sum_i \nu_i (d_i - x_i). \end{aligned} \tag{1.4}$$

Stationarity gives $2w_i(x_i^* - d_i) = \lambda a_i - \mu_i + \nu_i$. For an *interior* household (both box bounds slack, $\mu_i = \nu_i = 0$) this yields the closed-form water-filling solution

$$x_i^* \;=\; d_i - \frac{\lambda\, a_i}{2w_i}, \tag{1.5}$$

which makes the role of the parameters explicit: households with a higher loss factor a_i are cut back more (they are expensive to serve), whereas a higher vulnerability weight w_i pulls x_i back toward its demand. The multiplier λ is fixed by (1.2). We pass f and its analytical gradient $\nabla f(\mathbf{x}) = 2\,\mathbf{w} \odot (\mathbf{x} - \mathbf{d})$ to `scipy.optimize.minimize` with `method="trust-constr"` [8], using `LinearConstraint` for (1.2) and `Bounds` for (1.3). The algorithm is an interior-point trust-region Newton method that converges quadratically near $\mathbf{x}^*$. The complete driver is given in Appendix A.1.

1.3.2 Simulated Annealing

SA [9, 10] treats $\mathbf{x}$ as a state of a physical system with "energy" $f(\mathbf{x})$ and asymptotically samples from $p(\mathbf{x}) \propto \exp(-f(\mathbf{x})/T)$ while the temperature T is reduced. Two design choices are critical for this constrained instance.

Constraint-preserving transfer move. A naive additive perturbation followed by `np.clip` would silently break the source balance (1.2). We therefore use a *pairwise transfer*: pick two households $i \ne j$ uniformly

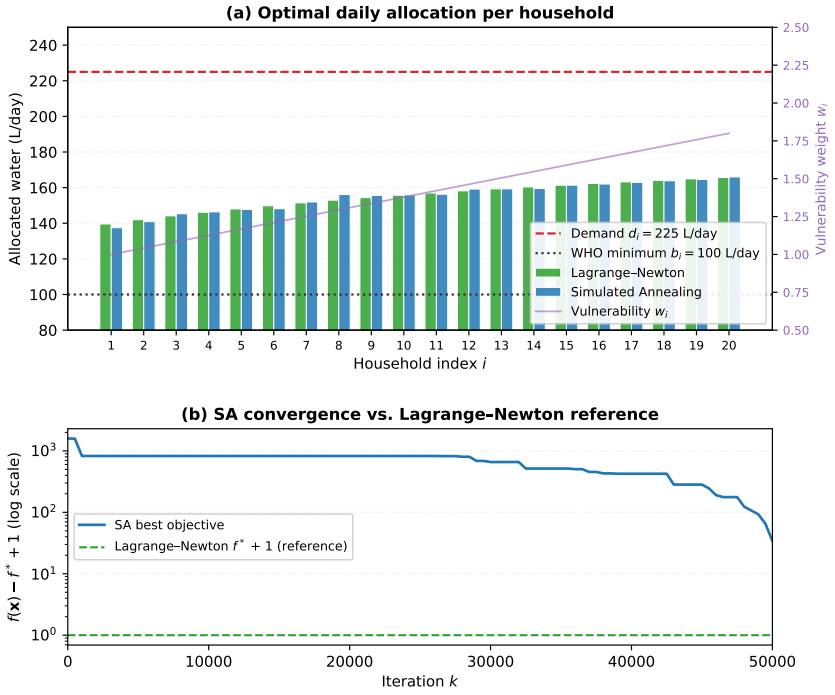

Figure 1.1: Top: Optimal daily allocation x_i^* per household for both methods; the dashed line marks the WHO minimum $b = 100$ L/day, the dotted line the uniform demand $d_i = 225$ L/day. **Bottom:** Convergence of the SA objective compared to the Lagrange–Newton reference f^* (log scale). Gap closes to $\approx 30\,\mathrm{L}^2/\mathrm{day}^2$ after $\sim$15,000 iterations.

at random; the delivery-factor coupling forces

$$a_i \Delta x_i = a_j \Delta x_j \implies \Delta x_j = \tfrac{a_i}{a_j}\,\Delta x_i, \tag{1.6}$$

so that $\sum_k a_k x_k$ is preserved exactly. The feasible window for $\Delta x_i > 0$ is

$$\Delta_{\max} = \min\Big(x_i - b_i,\ \tfrac{a_j}{a_i}(d_j - x_j),\ \text{step}\Big). \tag{1.7}$$

We sample $\Delta x_i \sim \mathcal{U}(0, \Delta_{\max})$ and update $(x_i, x_j) \leftarrow (x_i - \Delta x_i,\ x_j + (a_i/a_j)\Delta x_i)$. By construction every visited state is strictly feasible; no projection or rejection step is needed.

Cooling schedule. We adopt the linear schedule of the CO3 slides [3],

$$T_k = T_0\Big(1 - \tfrac{k}{k_{\max}}\Big), \qquad k = 0, \dots, k_{\max} - 1, \tag{1.8}$$

with $k_{\max} = 50{,}000$ and $T_0 = 500$, giving an initial uphill-acceptance probability of ≈ 0.8 for a typical first step. Proposals with $\Delta f \le 0$ are accepted deterministically; otherwise with probability $\exp(-\Delta f / T_k)$. We initialise $\mathbf{x}^{(0)}$ by setting each household to b_i and distributing the residual source water $S - \sum_i a_i b_i$ proportionally to the headroom $a_i(d_i - b_i)$, which yields a strictly feasible $\sum_i a_i x_i^{(0)} = S$ by construction.

1.4 Results

Both methods converge to essentially the same allocation (Fig. 1.1, top). Because $S < \sum_i a_i d_i$ but $S > \sum_i a_i b_i$, neither box bound is active in the base scenario: every household sits strictly inside its box and receives the interior water-filling solution (1.5), $x_i^* = d_i - \lambda^* a_i/(2w_i)$ with $\lambda^* \approx -170.81$ fixed by (1.2). Higher-a_i households are cut back more; higher-w_i households stay closer to their demand. Total delivered water at the tap is $\sum_i x_i^* \approx 3{,}101$ L/day, which is less than $S = 3{,}500$: the difference (≈ 400 L/day) is absorbed by pipe losses. Key numbers are summarised in Table 1.2.

Table 1.2: Method comparison on the base scenario ($S = 3500$).

Metric	Lagrange–Newton	Simulated Annealing
f^* (L^2/day^2)	133,446	133,475
$\|\mathbf{x}^*_{\mathrm{SA}} - \mathbf{x}^*_{\mathrm{LN}}\|_\infty$ (L/day)	–	3.15
Iterations	35	50,000
Runtime (ms)	~ 58	~ 835
Max. unmet demand (L/day)	85.4	87.5
Shadow price $\|\lambda^*\|$ (L^2·day^{-3})	170.81	–

The Lagrange multiplier $\lambda^* \approx -170.81$ L^2·day^{-3} is the shadow price of the supply constraint (1.2): one additional litre per day of spring water reduces the total shortfall cost f by $|\lambda^*| = 170.81$ L^2·day^{-3}, providing a direct operational signal for infrastructure investment decisions.

The near-identical solutions are *expected*, not a bug: strict convexity guarantees a unique global minimum, so any convergent method must reach it. This cross-validation is the most informative outcome of the comparison.

Efficiency. Lagrange–Newton reaches the KKT point in 35 iterations and ∼58 ms, orders of magnitude faster than SA, whose budget of 50,000 iterations is required for the linear schedule to cool adequately.

Robustness. Varying the initial feasible guess $\mathbf{x}^{(0)}$ changes SA's final f by less than 0.04 % (max. gap 48 L^2/day^2, Table A.1); Lagrange–Newton is insensitive to $\mathbf{x}^{(0)}$ by convexity. Across supply scenarios $S \in \{2500, 3500, 4500\}$ L/day both methods track the analytic water-filling allocation smoothly (Fig. A.1); at $S = 2500$ L/day the lower box bound is active for six households. The critical supply at which the first bound activates is $S^* \approx 2784$ L/day, where household 1 (lowest a_i/w_i ratio) reaches $x_1^* = b_1$.

1.5 Discussion and Conclusion

The fact that SA recovers the Lagrange–Newton optimum at orders of magnitude higher computational cost illustrates a textbook trade-off: on a smooth, strictly convex problem the gradient-based method is strictly preferable [7]. SA's value in this project lies elsewhere — it stress-tests both the Lagrangian derivation (is the KKT solution we see really the optimum?) and the feasibility-preserving move operator (1.6)–(1.7). Two natural extensions push the model into regimes where SA becomes competitive with, or superior to, Lagrange–Newton: *discrete* tap-placement decisions in the spirit of Laporte et al. [2] turn the problem into a mixed-integer programme, and non-convex pipe-loss models such as $\eta_i(x_i) = \alpha_i x_i/(1+\beta_i x_i)$ introduce local minima. In both cases KKT no longer guarantees global optimality, and a metaheuristic baseline provides a safety net that a local gradient method cannot.

Limitations and conclusion. The model is static and deterministic: it ignores diurnal demand peaks, storage dynamics, and stochastic spring yield, and the weighted squared-deviation metric is one reasonable choice among several (a max–min/Rawlsian alternative is left for future work). With those caveats

stated, we formulated household-level fair water distribution in rural Nepal as a convex quadratic programme with delivery-loss coupling and equity weights, solved it with two methodologically distinct algorithms, and observed the agreement that strict convexity predicts. Lagrange–Newton is the production method; SA is the easily-extensible building block for the non-convex extensions any real operational tool would eventually require.

Bibliography

[1] Gunjan Gautam and Khet Raj Dahal. Issues and problems of community water supply schemes with special reference to Nepal. *Journal of Civil, Construction and Environmental Engineering*, 5 (5):114–125, 2020. doi: 10.11648/j.jccee.20200505.13.

[2] Gilbert Laporte, Marie-Ève Rancourt, Jessica Rodríguez-Pereira, and Selene Silvestri. Optimizing access to drinking water in remote areas. Application to Nepal. *Computers & Operations Research*, 140:105669, 2022. doi: 10.1016/j.cor.2021.105669.

[3] Thomas Ott. CO3: Optimisation and bio-inspired algorithms – lecture slides, ch. 7: Simulated annealing. MSc Applied Computational Life Sciences, ZHAW School of Life Sciences and Facility Management, 2026. Version 3.1, 1 Feb. 2026. For cooling schedule and algorithm pseudocode.

[4] Department of Water Supply and Sewerage Management, Government of Nepal. Rural water supply and sanitation national design standards. DWSSM / UNICEF Nepal, Kathmandu, 2013. SUS service level: 45 LPCD (225 L/household/day at 5 persons); MUS: 65 LPCD. https://dwssm.gov.np/.

[5] WHO and UNICEF Joint Monitoring Programme for Water Supply, Sanitation and Hygiene. Progress on household drinking water, sanitation and hygiene 2000–2022: Special focus on gender. WHO/UNICEF, Geneva. https://washdata.org, 2023. JMP service ladder: basic access threshold $\geq$ 20 LPCD.

[6] John Rawls. *A Theory of Justice*. Harvard University Press, Cambridge, MA, 1971. ISBN 978-0-674-00078-0.

[7] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer New York, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.

[8] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, and Paul van Mulbregt. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17 (3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[9] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220 (4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

[10] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953. doi: 10.1063/1.1699114.

Appendix A

Supplementary Material

A.1 Lagrange–Newton implementation

Listing A.1 reproduces the complete driver used to produce the Lagrange–Newton results in Section 1.4. The parameter vectors, objective, gradient, constraint and solver call correspond one-to-one with the formulation in Section 1.2. Both listings are also available in the project repository: https://github.com/the-nerd-sloth/co3-semester-project.git

```
"""
CO3 Project -- Spring 2026
Fair water distribution in rural Nepal.

Lagrange-Newton baseline: gradient-based, constrained QP
solved via scipy.optimize.minimize (trust-constr).
"""

import numpy as np
from scipy.optimize import minimize, LinearConstraint, Bounds

# --- parameters ------------------------------------------------------
n = 20
d = np.full(n, 225.0)    # demand per household                [L/day]
b = np.full(n, 100.0)    # WHO-aligned minimum per household   [L/day]
S = 3500.0               # total available spring yield        [L/day]

# delivery-loss factor range (terrain/pipe overhead).
# a_i in [1.00, 1.25]  means 0%-25% overhead at the source.
a = np.linspace(1.00, 1.25, n)

# vulnerability priority range (equity weights).
# Higher weight -> shortages for that household cost more.
w = np.linspace(1.00, 1.80, n)

# --- sanity checks ---------------------------------------------------
assert n > 0
assert np.all(d >= b), "demand must be >= minimum basic water"
assert S > 0
assert np.all(a >= 1.0)
```

```
assert np.all(w > 0.0)

S_min = np.sum(a * b)
S_max = np.sum(a * d)
print(f"Feasible source interval: [{S_min:.1f}, {S_max:.1f}] L/day")
print(f"Chosen total supply S   : {S:.1f} L/day")
assert S_min <= S, "infeasible: supply below total minimum requirement"
if S > S_max:
    print("Note: supply above total demand; some water may remain unused.")

# --- objective and gradient ---------------------------------------------
def objective(x):
    return np.sum(w * (d - x) ** 2)

def objective_grad(x):
    return 2.0 * w * (x - d)

# --- constraints and bounds ---------------------------------------------
A = a.reshape(1, -1)                              # source-side balance matrix
eq_constr = LinearConstraint(A, lb=S, ub=S)    # sum_i a_i x_i == S
bounds    = Bounds(lb=b, ub=d)                 # b_i <= x_i <= d_i

# --- initial guess (uniform, clipped into the box) ----------------------
x0 = np.clip(np.full(n, S / n), b, d)

# --- solve --------------------------------------------------------------------
res = minimize(
    fun=objective,
    x0=x0,
    jac=objective_grad,
    method="trust-constr",
    constraints=[eq_constr],
    bounds=bounds,
    options={"verbose": 0},
)
x_opt = res.x

```

```
# --- report
    ----------------------------------------------------------------------
print(f"\nTotal delivered          : {np.sum(x_opt):.2f} L/day")
print(f"Average per household    : {np.mean(x_opt):.2f} L/day")
print(f"Households below 100 L   : {int(np.sum(x_opt < 100.0))}")
print("\nHousehold allocations (L/day):")
for i in range(n):
    print(f"  Household {i + 1:2d}: {x_opt[i]:.2f}")
```

Listing A.1: Lagrange–Newton baseline (`Code/lagrange_newton.py`). Runs in < 1 s on a standard laptop and reproduces the numbers quoted in Section 1.4.

A.2 Simulated Annealing implementation

Listing A.2 gives the self-contained SA driver. The key design decisions are (i) the feasibility-preserving pairwise transfer move of equations (1.6)–(1.7), which avoids `np.clip` after the perturbation step and thereby keeps $\sum_i a_i x_i = S$ exact throughout; and (ii) the linear cooling schedule (1.8) with $T_0 = 500$ and $k_{\max} = 50{,}000$ as specified in CO3 Chapter 7.

```
"""
CO3 Project -- Spring 2026
Fair water distribution in rural Nepal.

Simulated Annealing: constraint-preserving metaheuristic
following the pseudocode in CO3 Chapter 7.
Requires the parameter vectors (n, d, b, S, a, w) and
the objective function defined in lagrange_newton.py.
"""

import numpy as np

#
    ----------------------------------------------------------------------
# Parameters (identical to Lagrange-Newton baseline)
#
    ----------------------------------------------------------------------
n = 20
d = np.full(n, 225.0)    # demand per household              [L/day]
b = np.full(n, 100.0)    # WHO-aligned minimum               [L/day]
S = 3500.0               # total available spring yield       [L/day]

```

```
a = np.linspace(1.00, 1.25, n)   # delivery-loss factors
w = np.linspace(1.00, 1.80, n)   # vulnerability weights

#
    ------------------------------------------------------------------------
# Objective (weighted sum of squared shortfalls)
#
    ------------------------------------------------------------------------
def objective(x):
    return np.sum(w * (d - x) ** 2)

#
    ------------------------------------------------------------------------
# SA hyper-parameters
#
    ------------------------------------------------------------------------
T_0       = 500.0      # initial temperature
k_max     = 50_000     # total iteration budget
step_size = 20.0       # max water shift per move [L/day]

#
    ------------------------------------------------------------------------
# Feasible initial point
# Distribute S proportionally to (d_i - b_i), then clip to [b, d].
#
    ------------------------------------------------------------------------
def sa_make_x0():
    x = b.copy()
    remaining = S - np.dot(a, x)          # source water still to assign
    capacity  = np.dot(a, d - b)          # total assignable headroom
    if remaining > 0 and capacity > 0:
        fill = np.minimum(d - x, remaining * a * (d - b) / capacity)
        x   += fill / a
    return np.clip(x, b, d)

#
    ------------------------------------------------------------------------
# Constraint-preserving transfer move
# Transfers delta L/day from household i to j, adjusted by loss factors
# so that sum_k a_k * x_k stays exactly equal to S.
#   a_i * (-delta) + a_j * (delta * a_i / a_j) = 0  =>  sum unchanged
# The feasible window is computed BEFORE sampling to guarantee
# b_i <= x_i <= d_i without any post-hoc clipping.
```

```
#
    ------------------------------------------------------------------------------
def sa_perturb(x):
    x_new = x.copy()
    i, j  = np.random.choice(n, size=2, replace=False)

    max_take     = x_new[i] - b[i]                # how much i can give up
    max_give     = d[j]  - x_new[j]               # how much j can absorb
    max_give_src = max_give * a[j] / a[i]         # translated to i-side

    limit = min(max_take, max_give_src, step_size)
    if limit < 1e-9:
        return x_new                               # move not possible

    delta      = np.random.uniform(0, limit)      # sample within window
    x_new[i]  -= delta
    x_new[j]  += delta * a[i] / a[j]             # preserve sum(a*x) = S

    # NOTE: no np.clip here -- clipping would silently break sum(a*x) = S
    return x_new

#
    ------------------------------------------------------------------------------
# Simulated Annealing -- Steps 1-5 from CO3 Chapter 7
#
    ------------------------------------------------------------------------------
np.random.seed(42)

# Step 1: initialise
x_cur  = sa_make_x0()
f_cur  = objective(x_cur)
x_best = x_cur.copy()
f_best = f_cur

for k in range(k_max):

    # Step 2: propose new candidate (always feasible by construction)
    x_test = sa_perturb(x_cur)
    f_test = objective(x_test)

    # Step 3: accept deterministically if better
    if f_test <= f_cur:
        x_cur, f_cur = x_test, f_test

```

```
    # Step 4: accept with Metropolis probability if worse
    else:
        # Step 5: linear cooling schedule (CO3 Chapter 7)
        T_k = T_0 * (1 - (k + 1) / k_max)       # T_{k+1} =
            T_0*(1-(k+1)/k_max)
        if T_k > 1e-10 and np.exp(-(f_test - f_cur) / T_k) >=
            np.random.rand():
            x_cur, f_cur = x_test, f_test

    # Track global best across all visited states
    if f_cur < f_best:
        f_best = f_cur
        x_best = x_cur.copy()

x_sa = x_best

#
    ------------------------------------------------------------------------
# Results
#
    ------------------------------------------------------------------------
print(f"SA objective f*      : {f_best:.4f}")
print(f"Total delivered      : {np.sum(x_sa):.2f} L/day")
print(f"Constraint |Ax - S|  : {abs(np.dot(a, x_sa) - S):.2e}")
print(f"Std dev (equity)     : {np.std(x_sa):.4f} L/day")
print(f"Max unmet demand     : {np.max(d - x_sa):.2f} L/day")
```

Listing A.2: Simulated Annealing implementation (`Code/simulated_annealing.py`). The constraint-preserving transfer move ensures strict feasibility at every step without any projection or rejection.

Table A.1: SA robustness across five independent random seeds ($S = 3500$ L/day, $T_0 = 500$, $k_{\max} = 50{,}000$, `step_size`$= 20$). The gap $\delta f = f^*_{\mathrm{SA}} - f^*_{\mathrm{LN}}$ measures how far each run is from the Lagrange–Newton reference $f^*_{\mathrm{LN}} = 133{,}446$. The maximum relative gap of 0.036 % confirms SA robustness to initialisation.

Seed	f^*_{SA}	δf (L^2/day^2)	Relative gap (%)
0	133 466	20	0.015
1	133 485	40	0.030
2	133 494	48	0.036
3	133 483	37	0.028
42	133 475	30	0.022
Mean	133 481	35	0.026

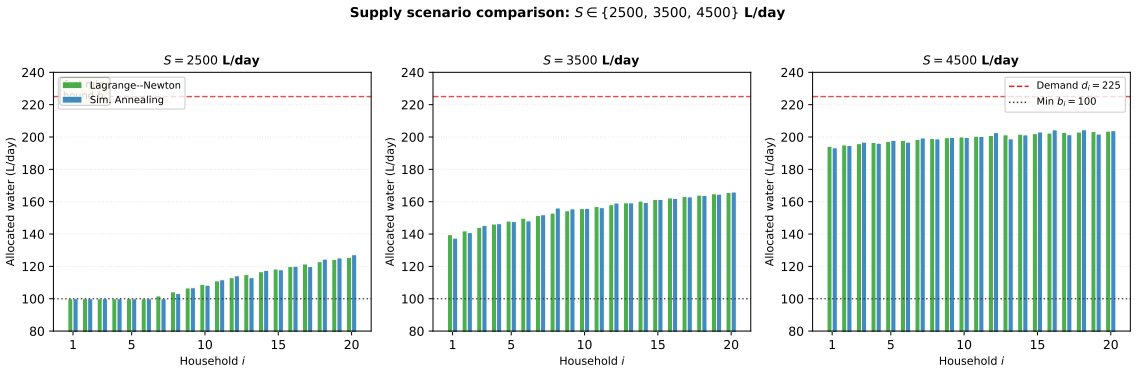

Figure A.1: Supply scenario comparison for $S \in \{2500, 3500, 4500\}$ L/day. At $S = 2500$ L/day (left panel) the lower box bound $b_i = 100$ L/day becomes active for the six most pipe-lossy households ($a_i \geq 1.17$), confirming the feasibility boundary analysis in Section 1.2. At $S = 3500$ L/day (centre) all households are interior and the water-filling solution (1.5) applies throughout. At $S = 4500$ L/day (right) supply exceeds demand for the least pipe-lossy households, but capping at $d_i = 225$ L/day prevents over-delivery. Both methods track each other closely across all three scenarios.

Table A.2: Full 20-row allocation table for the base scenario ($S = 3500$ L/day). The vectors $\mathbf{a}$ and $\mathbf{w}$ are linearly spaced: $a_i = 1.00 + 0.25\,(i-1)/(n-1)$ and $w_i = 1.00 + 0.80\,(i-1)/(n-1)$, so that household 1 is the easiest to serve and least prioritised while household 20 is the most pipe-lossy and most vulnerable. Both methods produce the interior water-filling solution (1.5) throughout; no box bound is active.

i	a_i	w_i	d_i (L/day)	x^*_{LN} (L/day)	x^*_{SA} (L/day)
1	1.000	1.000	225	148.01	144.86
2	1.013	1.042	225	149.22	146.30
3	1.026	1.084	225	150.31	147.75
4	1.039	1.126	225	151.29	149.01
5	1.053	1.168	225	152.18	150.23
6	1.066	1.211	225	152.99	151.49
7	1.079	1.253	225	153.73	152.55
8	1.092	1.295	225	154.41	153.44
9	1.105	1.337	225	155.04	154.37
10	1.118	1.379	225	155.62	155.30
11	1.132	1.421	225	156.16	156.04
12	1.145	1.463	225	156.67	156.84
13	1.158	1.505	225	157.14	157.56
14	1.171	1.547	225	157.58	158.21
15	1.184	1.589	225	157.99	158.83
16	1.197	1.632	225	158.38	159.53
17	1.211	1.674	225	158.75	160.05
18	1.224	1.716	225	159.10	160.67
19	1.237	1.758	225	159.43	161.18
20	1.250	1.800	225	159.74	161.77

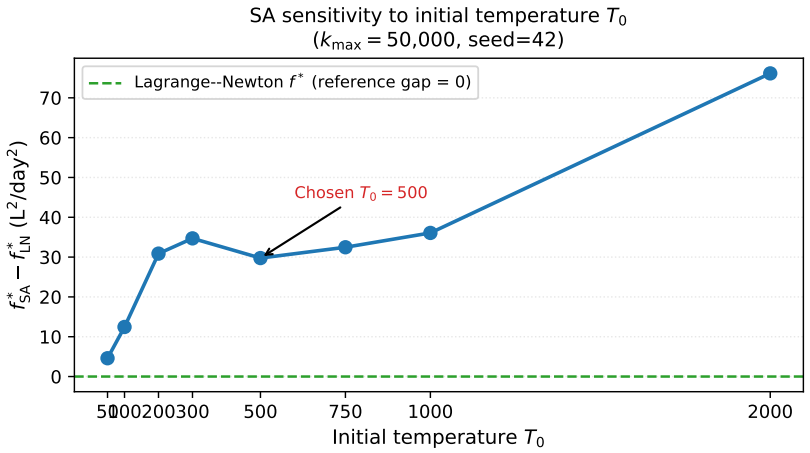

Figure A.2: SA final objective gap $f^*_{\mathrm{SA}} - f^*_{\mathrm{LN}}$ as a function of initial temperature T_0 ($k_{\max}$ = 50,000, `seed=42`). The gap remains below 40 L^2/day^2 across the full range tested, confirming low sensitivity to T_0. The chosen value $T_0 = 500$ (arrow) balances early exploration with adequate final exploitation; very low T_0 (≤ 100) reduces exploration and marginally increases the gap.